

من الكود إلى خدمة ذكاء اصطناعي قابلة للتشغيل والتطوير

شرح مشروع Mizan لمساعد القانون المدني المصري: كيف تشغله، وكيف
تعرض DVC و CI/CD، وما الذي يفعله كل جزء، وما الذي يلزم قبل وصفه بأنه
جاهز للإنتاج.

مبني على ملفات المشروع الحالية ومتطلبات Mini Project 2 في كتيب MLOps Practitioner. يفرّق الدليل بين ما تُقدّ فعلياً وما لا يزال يحتاج إعداداً
أو خدمة خارجية.

كيف أشغل التطبيق الآن؟

الواجهة الأمامية والـ API يعملان داخل تطبيق FastAPI واحد. قبل التشغيل يجب أن تتوفر البيانات والفهرس المحليان، وأن يكون نموذج BGE-M3 في ذاكرة Hugging Face، وأن يكون مفتاح Groq صالحاً لتوليد الإجابات.

أسرع طريقة: PowerShell من مجلد المشروع

```
# Copy .env.example once and set GROQ_API_KEY in .env
Copy-Item .env.example .env
# If .env already exists, keep it

.\.venv\Scripts\Activate.ps1
pip install -e ".[dev,pipeline,evaluation]"
dvc status
dvc repro
uvicorn src.api.main:app --reload
```

افتح `http://localhost:8000/health` الفحص: `/health` والوثائق التفاعلية: `./docs`.

ماذا يحدث؟

`dvc repro` يحضر المقالات والفهرس. Uvicorn يشغل الواجهة وAPI. البحث بالمتجهات محلي، ثم يرسل التطبيق السؤال والمقاطع المختارة إلى Groq لتوليد الإجابة.

المتطلبات

Python 3.10+، ومخبأ DVC محليان، فهرس Qdrant ناتج عن PDF، pipeline، نموذج BGE-M3 موجود محلياً، ومفتاح `GROQ_API_KEY` في `..env`.

التشغيل بحاوية Docker

```
docker compose up --build
# Open the UI at http://localhost:8000
# Stop with Ctrl+C, then
docker compose down
```

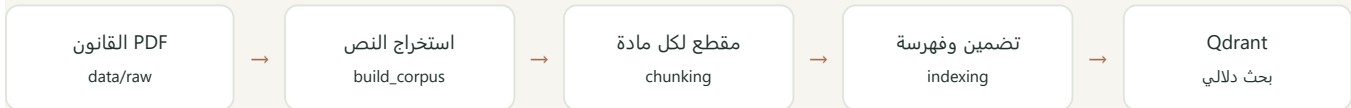
خطأ `dockerDesktopLinuxEngine`: هذه رسالة عن محرك Docker Desktop غير العامل؛ لم يصل Docker إلى مرحلة بناء `Dockerfile`. افتح Docker Desktop وانتظر جاهزية `Linux engine`، ثم تحقق بـ `docker info` و `docker compose version`. وأعد المحاولة. صورة المشروع مبنية على `Linux`.

سجل التنزيل الذي أرسلته يظهر عجلة PyTorch بحجم 554.6 MB. هذا يشرح جزءاً كبيراً من وقت التنزيل بسبب Sentence Transformers، لكنه لا يعرض وحده نهاية سجل الخطأ. فصلت أدوات DVC و MLflow و RAGAS عن image الخدمة؛ PyTorch يظل ضرورياً لـ BGE-M3.

١ · التشغيل قبل الشرح

DVC: كيف أعرض خط البيانات؟

DVC يصف مراحل إنتاج المقالات والفهرس ويربطها بالمدخلات والكود والمعاملات. `dvc.lock` يسجل بصمات آخر تشغيل؛ ويتيح مخبأ DVC إعادة استخدام النواتج عندما لا تتغير المدخلات.



```
# Show pipeline stages
dvc dag
# Check changed inputs and outputs
dvc status
# Run stale stages
dvc repro
# Repeat unchanged to demonstrate cache
dvc repro
```

المرحلة	ماذا تفعل؟	فائدتها
build_corpus	يفصل عمودي PDF العربي والإنجليزي وينشئ سجلاً لكل مادة.	استخراج قابل للإعادة ومراجعة فجوات المصدر.
chunking	ينشئ مقطعاً لكل مادة ويسجل رقمها ونصها وحالة إلغائها.	يحافظ على سياق المادة ويجعل الاستشهاد أوضح.
indexing	يحسب متجهات BGE-M3 محلياً ويكتبها إلى Qdrant.	يهيئ البحث الدلالي دون تنزيل النموذج عند كل تشغيل.

الحد الحالي: لا يوجد remote مشترك لـ DVC. يعمل `dvc repro` على الجهاز الذي لديه المصدر ومخبأ DVC، لكن clone جديد لا يستطيع استعادة البيانات بـ `dvc pull`. كذلك لا تعرّف المراحل ملفات `metrics`، لذا `dvc metrics show/diff` غير جاهز للمقارنة. لا تضع بيانات اعتماد التخزين في Git.

العرض العملي: اعرض `dvc dag` ثم `dvc status`، وشغّل `dvc repro` مرتين لتوضيح رسائل `cache`. لإثبات النقل بين جهازين، اختر remote خاصاً وسجّل إعداداته الآمنة ثم نفذ `dvc push` و `dvc pull`.

يحتفظ DVC بالبيانات والنواتج بعيداً عن تاريخ Git، فيما يحفظ Git تعريف المراحل وبصمات الملفات. يلزم remote موثوق لإتاحة النسخ لجهاز آخر.

CI/CD: كيف أعرضه؟

أضفت `.github/workflows/ci-cd.yml` عند Pull Request إلى `main` أو `push` إليه، تُفحص جودة الكود وتُجرى الاختبارات ثم تُبنى صورة Docker. النشر إلى Docker Hub يحصل بعد نجاح الفحوص على `main`.

CD • تسليم صورة

يبنى `image` بعد نجاح الفحوص. Pull Request يبنى دون نشر. `push` إلى `main` ينشر وسم `commit` ووسم `latest`. النشر إلى سجل الصور وليس إلى خادم إنتاج.

CI • تحقق مستمر

يثبت التطبيق وأدوات التطوير، يشغل `Ruff` و `Black` واختبارات API والاسترجاع. الاختبارات تستخدم `mocks` ولا تحتاج `Groq` أو `Qdrant` عاملين. يحفظ تقرير التغطية ك `artifact`.

إعدادات Docker Hub في GitHub

١ افتح **Settings** → **Secrets and variables** → **Actions**

٢ أضف Repository variable باسم `DOCKERHUB_USERNAME` وضع اسم مستخدمك.

٣ أضف Repository secret باسم `DOCKERHUB_TOKEN` وضع Access Token بصلاحيّة `push`. لا تكتبه في YAML.

٤ ارفع الملف إلى GitHub، افتح **Actions** ثم `CI and image delivery` لمشاهدة الوظائف والسجلات. اجعل نجاح الفحص شرطاً لدمج `main` من إعدادات `Branch protection`.

```
git add .github/workflows/ci-cd.yml Dockerfile docker-compose.yml pyproject.toml
git commit -m "Add CI checks and container delivery"
git push origin main
```

ما ليس آلياً: لا تشغل CI أمر `pull dvc` لأن `remote` غير معد، ولا تقييم `RAGAS` لأنه يحتاج `BGE` محلياً ومفتاح `Groq` واستدعاءات كثيرة. لا توجد بوابة تقارن مرشحاً بنموذج إنتاجي، ولا `continuous training` أو Terraform أو نشر لخدمة حية. عتبة `faithfulness` في برنامج التقييم يدوية.

للعرض: افتح Pull Request وأظهر `lint/test/build`. بعد إضافة بيانات الاعتماد، ادمج إلى `main` واعرض رابط الصورة والوسمين. لا تكشف السر في اللقطات.

ما الذي تغيّر في Dockerfile؟

الملف السابق كان يبني صورة واحدة ويثبت كل المكتبات دفعة واحدة، ومنها DVC و MLflow و RAGAS و LangChain، ثم ينسخ بيانات وفهرس الجهاز إلى image. خدمة API لا تستورد معظم أدوات التجارب، لذلك أصبحت هذه الحزم اختيارية.

التغيير	الفائدة
مرحلة builder ثم runtime	فصل تجهيز بيئة Python عن تشغيل التطبيق.
المستخدم mizan بدلاً من root	تقليل الصلاحيات التي تملكها عملية الخدمة.
تثبيت اعتماديات API فقط	عدم تثبيت DVC/RAGAS/MLflow داخل runtime: PyTorch باقي من أجل التضمين.
عدم نسخ Qdrant وملفات النموذج	يُركَّبان من host كمجلدات volumes قابلة للاستبدال.
Health check ومهلة بدء	إتاحة وقت لتحميل نموذج التضمين قبل إعلان توقف الخدمة.
.dockerignore	استبعاد البيانات و cache والتقارير من build context.

أوامر Compose:

```
docker compose up --build
docker compose ps
docker compose logs -f rag-api
docker compose down
```

يربط Compose qdrant_storage ومخبأ Hugging Face للقراءة. عدل مسار ال cache إذا كان نموذجك في مجلد مختلف.

حجم image: فصل الإضافات يقلل المكتبات غير اللازمة، لكن PyTorch الذي يظهر في سجلّك بحجم 554.6 MB مطلوب وقت التشغيل لتضمين BGE-M3. لا تستطيع multi-stage إزالة اعتماد runtime؛ قس الحجم بعد بناء ناجح بواسطة `docker image ls`.

شرط التشغيل: image لا تضم فهرس Qdrant ولا ملفات BGE-M3. جهز الفهرس على الجهاز وأتّح cache النموذج داخل المسار المرغّب. Docker يضمن بيئة التشغيل ولا ينشئ بيانات المشروع وحده.

بنية RAG في المشروع

المشروع لا يدرّب نموذج لغة من الصفر. بل يسترجع مواداً قانونية ذات صلة، ثم يطلب من نموذج توليد صياغة إجابة مبنية على تلك المواد.

سؤال المستخدم

FastAPI

محلي BGE-M3

Qdrant

مواد قانونية

مراجع + Groq

chunking · ٢

`src/rag/chunking.py` ينشئ مقطعاً لكل مادة مع رقم الاستشهاد والنص العربي وحالة الإلغاء؛ المواد 54-80 معلّمة ملغاة.

ingestion · ١

`src/ingestion/build_corpus.py` يفصل عمودي PDF، يلتقط أرقام المواد ويوحد بعض الأرقام والحروف. لا يخترع نصاً للمواد الغائبة.

retrieval · ٤

`src/rag/retriever.py` يضمّن السؤال ويعيد أعلى النتائج حسب `top_k`. البحث دلالي فقط؛ لا توجد كلمات مفتاحية أو BM25 أو hybrid.

indexing · ٣

`src/rag/indexing.py` يحمل BAAI/bge-m3 محلياً ويخزن متجهات Cosine في Qdrant.

API · ٦ والواجهة

`src/api/main.py` يقدم `POST /ask` و `GET /health` ويستضيف ملفات HTML/CSS/JS. كل شيء من أصل واحد.

generation · ٥

`src/rag/generator.py` يرسل السؤال والنصوص المسترجعة إلى Groq، ويعيد إجابة ومراجع. الاستشهاد لا يضمن تفسيراً قانونياً صحيحاً.

تمييز مهم: BGE-M3 يعمل محلياً للبحث عن المواد. Groq يستضيف نموذج الإجابة؛ لذلك السؤال والمقاطع المختارة تُرسل خارج الجهاز. تجنب الأسئلة التي تحتوي بيانات شخصية أو سرية ما لم تسمح سياسة الخصوصية بذلك.

حدود النص: تقرير الجودة يوثق 1,096 مادة مستخرجة وفجوات نصية في PDF. راجع `reports/corpus-quality.md`؛ لا يمكن للاسترجاع العثور على نص غير موجود في `corpus`.

هيكل واضح وبسيط

```
data/
  raw/           DVC ومؤشر PDF
  processed/     corpus والمقاطع
  evaluation/    أسئلة التقييم
  qdrant_storage/ قاعدة المتجهات
  reports/       تقارير الجودة والتجارب
  params.yaml    نماذج ومسارات وإعدادات
  dvc.yaml       مراحل خط البيانات
  dvc.lock       بصمات آخر تنفيذ
  Dockerfile     صورة خدمة الإنتاج
  docker-compose.yml تشغيل الحاوية
  .github/workflows/
  ci-cd.yml      فحوص وتسليم الصورة
```

```
frontend/
  index.html     واجهة المحادثة
  styles.css     تصميم متجاوب
  app.js         إرسال وعرض النتائج
src/
  ingestion/     PDF إلى corpus
  rag/
    chunking.py  مقطع لكل مادة
    indexing.py  تضمين وإنشاء Qdrant
    retriever.py بحث دلالي
    generator.py توليد prompt Groq
  api/
    main.py      واستضافة الواجهة API
    schemas.py   التحقق من الطلبات
  experiments/
    evaluate_rag.py مقارنة إعدادات البحث
```

أهم إعدادات params.yaml

المفتاح	دوره	القيمة الحالية
embedding_model	نموذج التضمين المحلي.	BAAI/bge-m3
qdrant.path / collection_name	مجلد التخزين ومجموعة المتجهات.	./qdrant_storage / egyptian_civil_code
retriever.top_k	عدد المواد المرسلة للمولد.	يتأثر بإعداد التجربة الأفضل.
mlflow.*	مكان تتبع التجارب واسمها.	SQLite محلية.
generator.*	نموذج Groq وحرارته وحد التوكن.	openai/gpt-oss-120b عبر Groq.

حدود المسؤوليات: ingestion يجهز المصدر؛ RAG يبني المعرفة ويسترجعها؛ API تقدّم عقد HTTP ثابتاً؛ frontend تعرضه؛ DVC يتابع نواتج البيانات؛ MLflow يسجل التجارب.

كيف أستخدم MLflow وRAGAS؟

يقارن البرنامج خمس قيم لـ top_k على 20 سؤالاً. تقيس RAGAS أمانة الإجابة للسياق، ودقة السياق المسترجع وتغطيته، ومدى صلة الإجابة بالسؤال.

```
# والفهرس جاهزين env. من جذر المشروع مع
python -m src.experiments.evaluate_rag

# في طرفية ثانية
mlflow ui --backend-store-uri sqlite:///mlflow.db
# افتح http://localhost:5000
```

الخطوة	الناتج	الفائدة
تحميل الأسئلة والمراجع	20 سؤالاً مربوطاً بمادة حقيقية.	مجموعة ثابتة لإعادة القياس.
تشغيل كل top_k	إجابات وسياقات متعددة.	تحديد أثر عدد نتائج الاسترجاع.
مقاييس RAGAS	أربعة مقاييس لجودة RAG.	ملاحظة هل الإجابة مسنودة ومناسبة.
تسجيل MLflow	معلومات ومقاييس وملف أسئلة وملخص.	مقارنة النتائج والرجوع لها.
اختيار الأعلى faithfulness	عتبة 0.75 وalias باسم champion وتحديث top_k.	حفظ إعداد مرجعي للتجارب.

تكلفة وزمن: خمس قيم × 20 سؤالاً تعني 100 دورة توليد على الأقل، إضافةً إلى استدعاءات تقييم RAGAS. يلزم مفتاح Groq وقد يستهلك رصيد API. لا تشغله مع كل Pull Request دون ميزانية.

ما ليس موجوداً: MLflow يستخدم SQLite محلية لا PostgreSQL/Object Store. يُسجّل النموذج بصيغة PyFunc ويُنشأ alias champion، لكن API لا تحمّله من Registry؛ ما زالت تقرأ top_k من params.yaml. لا يوجد سير عمل None → Staging → Production ولا مقارنة مرشح بإنتاج.

قيمه للإنتاج: MLflow يوضح أثر إعداد البحث وأرقامه. النسخة الإنتاجية تحتاج تخزيناً مشتركاً متيناً، وبصمة Git/DVC لكل تشغيل، وسياسة ترقية محكمة.

أين يقف المشروع من Project 2؟

كتيب Mini Project 2 يصف خطأ عاماً لنماذج انحدار مدة الرحلة. مشروعنا مساعد RAG قانوني؛ لذلك نطبق أفكار التتبع وإعادة الإنتاج على الاسترجاع والتوليد، ولا نحقق كل متطلبات مقارنة وترقية نموذج الانحدار.

متطلب الكتيب	الحالة الحالية	ما ينقص
تسجيل تجارب متعددة	جزئي خمس إعدادات top_k ومقاييس RAGAS.	ليست +13 تجربة لثلاث عائلات نماذج وسويب 10 محاولات.
Model Registry	جزئي PyFunc و alias champion.	لا انتقال None → Staging → Production ولا تحميل API من Registry.
DVC + remote + cache + metrics	جزئي pipeline و lock و cache محلي.	remote و push/pull وملفات metrics وبصمة بيانات في MLflow.
CI جودة وبناء ونشر	جزئي Workflow يفحص ويختبر ويبني وينشر صورة.	بوابة مقارنة جودة، branch protection ولقطات النتائج؛ RAGAS ليس في CI.
Continuous Training	غير منفذ	جدولة و manual و repository_dispatch وتدريب وترقية Staging مع موافقة إنسان للإنتاج.
Terraform	غير منفذ	تعريف التخزين والبنية وتشغيل دورة plan/apply/destroy.
تقرير Module 2 ومراجعة PR	جزئي README وتقارير البيانات والتجارب.	تقرير الموديول ولقطات MLflow/CI ومراجعة زميل ووسم إصدار.

ما تم تنفيذه فعلاً: معالجة PDF، تضمين محلي، استرجاع دلالي، إجابات مع مراجع، FastAPI وواجهة ويب، DVC pipeline محلي، تقييم Docker، RAGAS/MLflow محسّن و CI لتسليم الصورة.

لماذا نوضح الفجوات؟ حتى يستطيع المقيم تشغيل ما هو موجود والتحقق منه، ودون خلط تجربة محلية بنظام تدريب وترقية ذاتي مكتمل. إعداد التخزين السحابي والأسرار يحتاج حسابات وبيانات اعتماد صاحب المشروع.

كيف يخدم المشروع MLOps للإنتاج؟

فحص قبل التسليم

CI يعيد التنسيق والاختبارات قبل بناء صورة جديدة.

إتاحة الخدمة

FastAPI يقدم HTTP؛ Docker يحزم بيئة تشغيل متماثلة للنقل.

تكرار البيانات

DVC يربط مصدر المقالات بمراحل التحويل؛ MLflow يحتفظ بأثر إعدادات البحث.

خطوات عملية لاحقة

- إعداد remote خاص لـ DVC، وتعريف metrics، وربط CI بنسخة بيانات يمكن استعادتها.
- اختبار جودة RAG ومقارنة المرشح بمرجع ثابت، وإيقاف الترقية عند تدهور النتائج.
- تخزين MLflow في Postgres/Object Store، وتسجيل commit وبصمة البيانات.
- إضافة مصادقة وحدود طلبات وسجلات منظمة ومقاييس latency/errors ونسخ احتياطي للفهرس.
- تثبيت dependencies وفحص الصورة أمنياً وتحديد خطة rollback وموافقة بشرية قبل التفعيل.

الحد القانوني: هذه أداة بحث تعليمية وليست استشارة قانونية. corpus به فجوات معلنة وقد يكون النص غير مكتمل. قد يخطئ النموذج؛ وجود استشهاد لا يثبت صحة التفسير. اعرض النص الأصلي واطلب مراجعة محام للمسائل الواقعية.

عرض مختصر للمقيّم

- ١ أسأل سؤالاً عربياً في الواجهة وافتح المرجع الظاهر.
- ٢ اشرح أن BGE-M3 محلي للبحث، و Groq خارجي للتوليد.
- ٣ اعرض dvc dag، ثم dvc repro مرتين ورسائل cache.
- ٤ اعرض Actions وبيّن حاجة Docker Hub variable/secret.
- ٥ اذكر بوضوح فجوات remote و quality gate و Terraform و CT.

المراجع الداخلية: README.md للتشغيل الكامل؛ reports/corpus-quality.md لجودة الاستخراج؛ dvc.lock و dvc.yaml للمراحل والبصمات؛ كتيب MLOps Practitioner، صفحات Module 2، الصفحات 18-24.